Operations on Stack

This chapter aims to demystify the main operations involved in manipulating a stack: push, pop, peek, and isEmpty. We will examine these operations closely, detailing their functionality, providing coding examples, and highlighting their importance in problem-solving.

Push Operation

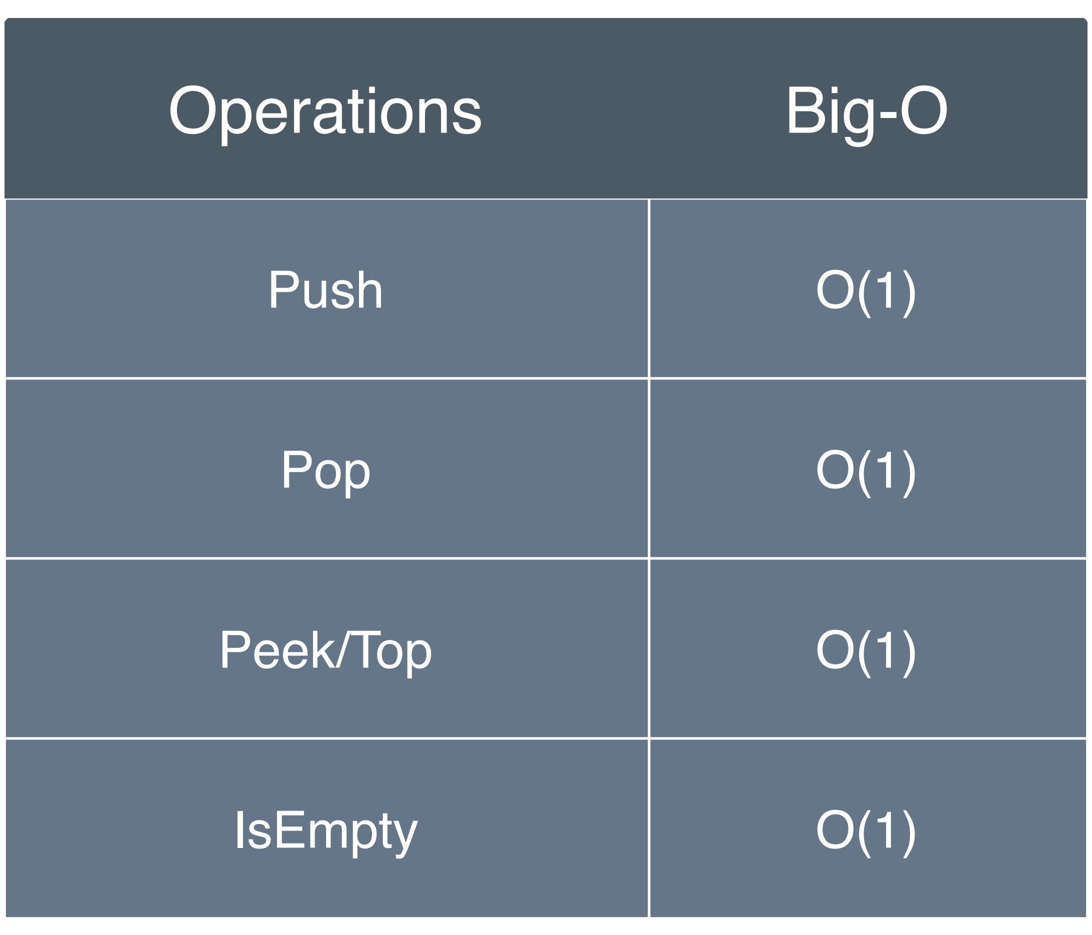
Let's start with the push operation. As we've previously learned, push adds a new element to the top of the stack. Think of it like placing a new dish on top of a pile in your sink - the new dish (our data) is added to the top of the pile (our stack).

In programming, the push operation usually involves a few steps. First, we check if there's room to add a new element (we'll discuss this in more detail when we talk about stack overflow). If there's room, we add the new element to the top of the stack.

Pop Operation

Next, we have the pop operation, which removes the topmost element of the stack. It's like removing the top dish from our pile in the sink.

In code, popping an element from a stack is usually done in two parts. First, we check if there are any elements to remove (we'll look at this more when we discuss stack underflow). If there are elements, we remove the top one.



Operations	Big-O
Push	O(1)
Pop	O(1)
Peek/Top	O(1)
IsEmpty	O(1)

Stack operations and their time complexities

Peek or Top Operation

The peek or top operation is slightly different. Instead of adding or removing an element, this operation allows us to look at the top element of the stack without removing it. It's like looking at the top dish in our sink pile without touching it.

This operation can be handy when you need to know what's on top of your stack, but you don't want to change anything. It's a read-only operation.

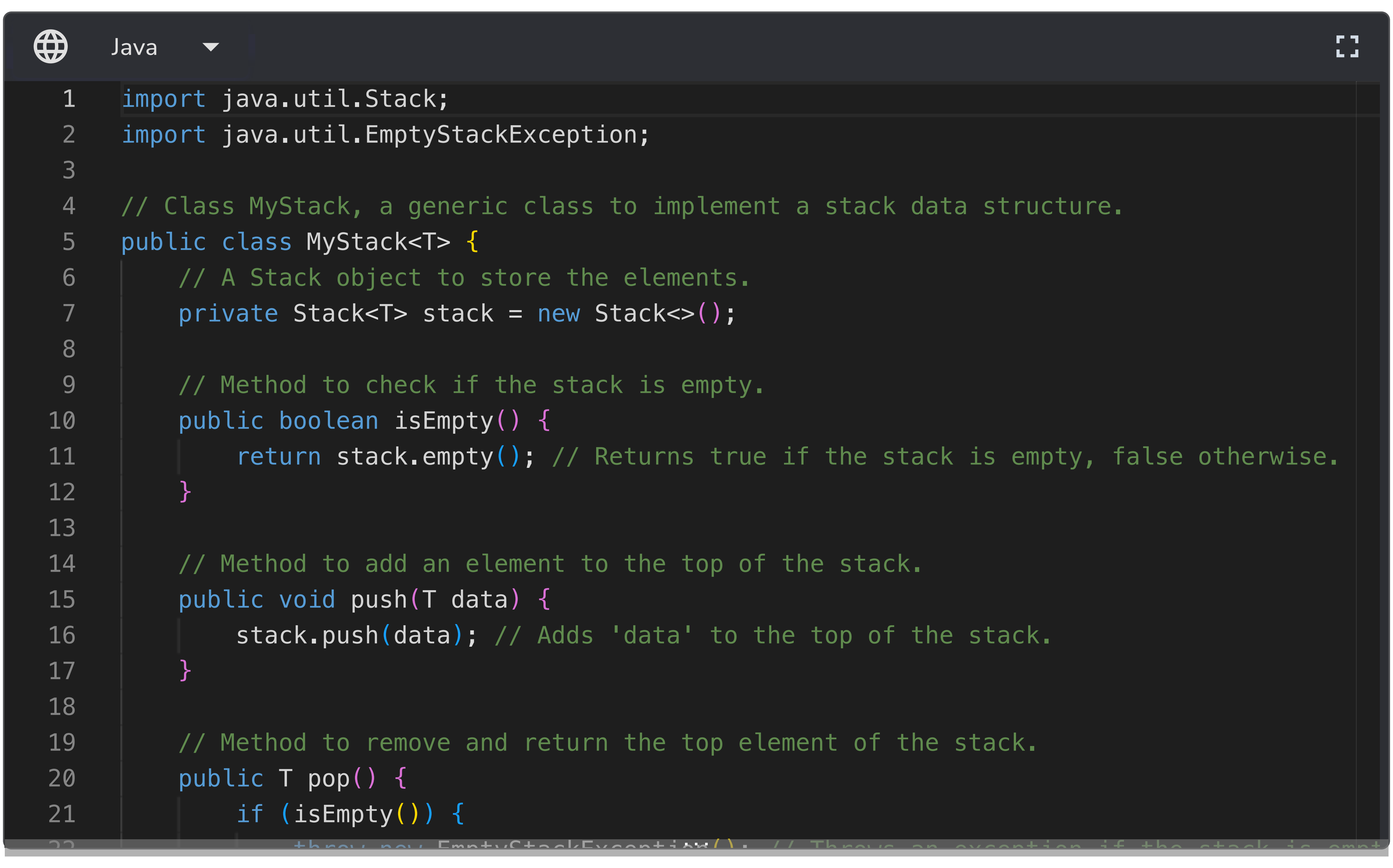
IsEmpty Operation

Finally, the isEmpty operation checks if the stack is empty. This operation is essential for preventing errors when popping an element from an empty stack (known as stack underflow).

An empty stack will return `true` when the isEmpty operation is performed, while a stack with one or more elements will return `false`.

Putting it All Together

Knowing how each of these operations works is crucial. But, what's more important is knowing how to use them together to solve problems. Here's a brief coding example to show how these operations can be used together.



```java
import java.util.Stack;
import java.util.EmptyStackException;

// Class MyStack, a generic class to implement a stack data structure.
public class MyStack<T> {
    // A Stack object to store the elements.
    private Stack<T> stack = new Stack<>();

    // Method to check if the stack is empty.
    public boolean isEmpty() {
        return stack.empty(); // Returns true if the stack is empty, false otherwise.
    }

    // Method to add an element to the top of the stack.
    public void push(T data) {
        stack.push(data); // Adds 'data' to the top of the stack.
    }

    // Method to remove and return the top element of the stack.
    public T pop() {
        if (isEmpty()) {
```

Dealing with Stack Overflow and Underflow

As we discussed earlier, stack overflow and underflow are two situations you might encounter when working with stacks. Stack overflow occurs when you try to push an element onto a stack that's already full, while stack underflow occurs when you try to pop an element from an empty stack.

Handling these situations properly is crucial to preventing runtime errors and ensuring that your code runs smoothly. Depending on the programming language and the specific implementation of the stack, you might have different ways of handling these situations. It's always a good idea to check for stack overflow and underflow before performing push and pop operations.

← Previous
Introduction To Stack

Next →
Implementing Stack Data Structure

☐ Mark as Completed